Object Oriented Programming Project

Typing Race Simulator

ECS414U

The quick brown fox jumps over the lazy dog...

Scenario

You have inherited a Java project from Ty Posaurus, a developer who typed exclusively with two fingers and considered that "perfectly adequate for professional work". He left the project in a state he described as "basically done", accompanied by a sticky note reading: "the slide-back thing is optional probably".

It is not optional.

Your task is to complete and improve this typing race simulation. The existing codebase provides two classes, Typist and TypingRace, available on the same QM+ page as these instructions. In a typing race, competitors race to type through a passage of text as quickly and accurately as possible. A competitor advances one character for each correct keystroke, but mistypes send them sliding backwards – and pushing too hard risks a burnout that freezes them for several turns.

The TypingRace class is partially functional (emphasis on "partially"), but contains errors that require fixing. The Typist class, however, is a skeleton. Ty got as far as writing the method signatures before apparently losing interest.

Typist Class

The Typist class represents individual competitors in the race. Each typist has:

- **Name & Symbol:** Every typist has a unique name (used for announcing the winner) and a single Unicode character symbol used for race visualisation (e.g. '①', '②', '③'). Each typist must have a different symbol so they can be told apart on screen.
- **Accuracy Rating:** A decimal number between 0 and 1 representing typing accuracy. Higher accuracy means faster progress but, paradoxically, also a greater risk of burnout from pushing too hard. Lower accuracy means slower but steadier progress.
- **Progress:** Unlike a simple race, a typist's progress can go **both forwards and backwards**, advancing on correct keystrokes and sliding back on mistypes. Progress cannot go below zero.
- **Burnout State:** A typist who burns out is frozen for a fixed number of turns, unable to type. Unlike a permanent failure, burnout is **temporary**, i.e., the typist recovers and re-enters the race.
- **Performance Adjustment:** Winning a race slightly increases a typist's accuracy. Burning out slightly decreases it.

TypingRace Class

The TypingRace class manages the simulation using Typist objects and provides a command-line animated display of the race.

- **Race Configuration:** The passage length (the number of characters in the passage) is configurable and is represented simply as an integer (argument to the TypingRace constructor). For Part I, the actual text content of the passage is not required; progress is purely a number advancing from 0 to `passageLength`. Think of it exactly like the track length in a conventional race.
- **Race Mechanics:** Each turn, typists may advance, slide back on a mistype, or burn out. The interplay of these mechanics creates a dynamic, unpredictable race.
- **Race Completion:** The race ends when the first typist's progress reaches or exceeds the passage length (i.e. `getProgress() >= passageLength`). Because progress can overshoot (for instance if a typist was already one character away and typed two) the win condition uses `>=` rather than `==`. The winner is then announced.

Part I: Textual Typing Race Simulator

Task 1: Write the Typist Class

Complete the Typist class to meet the following specifications. Note that this class is meaningfully different from a simple racer: read each point carefully.

1. The class should contain the following fields:
   a. The name of the typist (e.g. "TURBOFINGERS")
   b. A single Unicode character that represents the typist on screen (e.g. '①', '②', '③')
   c. The typist's current progress through the passage, stored as a whole number (can go up and down, but never below zero)
   d. A Boolean flag indicating whether the typist is currently burnt out
   e. The number of turns of burnout remaining (an integer, zero when not burnt out)
   f. The accuracy rating, a decimal number between 0.0 and 1.0
2. The constructor should have the following signature:

```
public Typist(char typistSymbol, String typistName, double typistAccuracy)
```

3. The class should provide the following public methods:

Method	Description
`burnOut(int turns)`	Sets the typist into a burnt-out state lasting the given number of turns.
`recoverFromBurnout()`	Reduces the burnout counter by one. When it reaches zero, the typist is no longer burnt out.
`getAccuracy()`	Returns the typist's accuracy rating.
`getProgress()`	Returns the typist's current progress (characters typed correctly so far).
`getName()`	Returns the name of the typist.
`getSymbol()`	Returns the character used to represent the typist.
`getBurnoutTurnsRemaining()`	Returns the number of turns of burnout remaining (0 if not burnt out).
`resetToStart()`	Resets progress to zero and clears all burnout state. Ready for a new race.
`isBurntOut()`	Returns true if the typist is currently burnt out, false otherwise.
`typeCharacter()`	Advances the typist forward by one character.
`slideBack(int amount)`	Moves the typist backwards by the given number of characters. Progress cannot go below zero.
`setAccuracy(double newAccuracy)`	Sets the accuracy rating. Values outside 0.0–1.0 are clamped to the valid range.
`setSymbol(char newSymbol)`	Sets the character used to represent the typist.

4. Encapsulation should be implemented to safeguard data in your class.

> *Important Note: It is crucial that the Typist class exactly matches the specifications above. Any deviation will cause compatibility issues with the TypingRace class left behind by Ty Posaurus, who, again, is unavailable to help debug this.*

Report Requirements:

- **Encapsulation Explanation:** Explain how encapsulation safeguards data in your Typist class. Identify which methods are accessors (getters) and which are mutators (setters). Pay particular attention to `slideBack()` and `setAccuracy()`. Explain the validation logic in each and why it matters.
- **Testing Evidence:** Include screenshots of your tests with detailed explanations. Your tests should cover:
  - That progress cannot go below zero after calling slideBack()
  - That burnout correctly counts down turn by turn and clears at zero
  - That resetToStart() clears both progress and burnout state
  - That accuracy cannot be set outside the 0.0–1.0 range
  - Normal forward movement via typeCharacter()

Task 2: Improve the TypingRace Class

Begin by examining the TypingRace class left behind by Ty Posaurus. It contains several deliberate errors and omissions. Your job is to find them, explain them, and fix as many as you can.

5. a) The **`startRace()`** method serves as the main entry point. Enhance it to display the name of the winning typist in the terminal once the race concludes. Below is a sample snapshot of what the race should look like mid-simulation:

```
  TYPING RACE - passage length: 40 chars
==========================================
|          ①                             | TURBOFINGERS   (Accuracy: 0.85)
|    ②  [<]                              | QWERTY_QUEEN   (Accuracy: 0.60) ← just mistyped
|               ③~                      | HUNT_N_PECK    (Accuracy: 0.30) BURNT OUT (2 turns)
==========================================
  [~] = burnt out     [<] = just mistyped

  And the winner is... TURBOFINGERS!
  Final accuracy: 0.87 (improved from 0.85)
```

6. b) Conduct thorough testing to identify bugs in the **`TypingRace`** class. Document every problem you find and attempt to resolve as many as possible. You will receive credit for identifying issues even if you are unsure of the fix. Propose solutions for everything you find.

In your report:

- List every identified issue with a clear explanation of what it causes (crash, wrong output, unfair behaviour, etc.).
- Provide updated code with explanations of every change made.
- For each bug, describe what a correct implementation should do instead.

> *Hint: think carefully about the win condition in raceFinishedBy(). When exactly should a typist be declared the winner? What happens if their progress overshoots the passage length – can that happen, and if so, does the current code handle it correctly?*

Part II: GUI Module Development

In this phase you will bring the typing race to life with a graphical user interface. The goal is a polished, interactive application that looks and feels nothing like a terminal, while keeping the same underlying simulation mechanics from Part I.

1. Interactive Race Configuration

Allow users to set up the race before it begins:

7. Passage Selection: Users can choose from a list of pre-defined passages of varying lengths (short/medium/long), or enter a custom passage. The passage length directly determines the track length. Unlike Part I where the passage is abstract, the GUI should display the actual text of the passage with each typist's cursor visually advancing through it character by character, similar to how real typing race applications such as TypeRacer work. Completed characters can be highlighted in a different colour to make progress immediately readable.
8. Seat Count: Users can configure how many typists compete (2-6).
9. Difficulty Modifiers: Users can apply global modifiers that affect all typists, such as:
    a. Autocorrect On/Off: When enabled, the slideBack amount is halved, simulating modern phone keyboards.
    b. Caffeine Mode: All typists gain a temporary speed boost for the first 10 turns, followed by increased burnout risk.
    c. Night Shift: Accuracy ratings are slightly reduced across the board: everyone is tired.

2. Customisable Typists

Give users rich options to personalise their competitors:

10. Typing Style: Choose from a predefined list (e.g. Touch Typist, Hunt & Peck, Phone Thumbs, Voice-to-Text). Each style influences the accuracy and burnout profile.
11. Keyboard Type: Select from different keyboards (e.g. Mechanical, Membrane, Touchscreen, Stenography). Each has different effects on speed and mistype rates.
12. Symbol & Colour: Users choose a custom character or emoji to represent the typist on screen, with a colour applied to their progress bar.
13. Accessories: Optional add-ons that affect performance, such as:
    a. Wrist Support: Reduces burnout duration.
    b. Energy Drink: Increases accuracy for the first half of the race, decreases it in the second half.
    c. Noise-Cancelling Headphones: Reduces the chance of a mistype.
14. Attribute Impact: Clearly specify how each customisation choice affects the typist's accuracy, speed, and burnout risk.

3. Statistics and Analytics

Track and display detailed performance data across races:

15. Performance Metrics: After every race, calculate and display for each typist:
    a. Words Per Minute (WPM): Calculated from passage length and time taken.
    b. Accuracy Percentage: Proportion of keystrokes that were correct (not mistypes).
    c. Burnout Count: Number of times the typist burnt out during the race.
    d. Accuracy Changes: How the typist's accuracy rating changed after the race.

16. Personal Bests: Maintain a record of each typist's best WPM across all races.
17. Historical Data: Store each typist's full race history (position, WPM, accuracy, burnout events) and display trends over time.
18. Comparison View: Allow users to compare two or more typists side by side on a chosen metric.
19. Graphical Display (optional): Include charts such as a WPM progression line graph or a bar chart of accuracy across races.

4. Reward System

Implement at least one of the following two options. Implementing both earns higher marks.

Option A: Leaderboard & Ranking System

After each race, award points based on finishing position (e.g. 3 pts for 1st, 2 pts for 2nd, 1 pt for 3rd). Track cumulative points across races and maintain a global leaderboard:

20. Points Algorithm: Calculate points per race accounting for finishing position, WPM achieved, and whether the typist burnt out.
21. Leaderboard Display: Show a running leaderboard ranked by cumulative points, updated after each race.
22. Titles & Badges: Award titles based on milestones (e.g. "Speed Demon" for 3 consecutive wins, "Iron Fingers" for completing 5 races without a burnout).
23. Rank Impact: Optionally, a typist's rank could influence their starting accuracy in the next race (champions face higher expectations).

Option B: Sponsor & Prize System

Typists earn virtual prize money based on their performance. Sponsors offer bonuses for specific achievements:

24. Earnings Algorithm: Base prize money per race scales with finishing position. Speed bonuses for WPM above a threshold. Burnout penalties reduce earnings.
25. Sponsor Deals: Each typist can be assigned a sponsor that offers a bonus for a specific condition (e.g. "KeyCorp: +50 coins if you finish without a single burnout").
26. Cumulative Earnings: Track total earnings across races and display a financial leaderboard.
27. Upgrades: Earnings can be spent on upgrades (better keyboard, wrist support) that improve attributes in future races.

Part III: Git Integration

Integrate Git into your project from the very beginning. Ty Posaurus did not use version control. His only backup strategy was "it's probably fine". You now know how that ended.

28. Initialise Git Repository: Initialise a local Git repository at the start of the project. Name the root folder TypingRaceSimulator.
29. Organise Project Structure:

```
TypingRaceSimulator/
│
├── .git/           # Git repository folder (hidden)
├── Part1/          # Code for Part I (textual version)
└── Part2/          # Code for Part II (graphical version)
```

30. Create GitHub Repository: Create a private repository on GitHub and push your local repository to it.
31. README File: Include a README.md in the root containing setup instructions, dependencies, and usage guidelines for both parts.
32. Commit Messages: Use descriptive, meaningful commit messages. "Fixed bug" is not a commit message. "Fixed mistype direction in advanceTypist() – was inverting accuracy" is a commit message.
33. Branching: Create a branch named gui-development for all Part II work, keeping the main branch stable.
34. Merging: After completing the GUI, merge gui-development back into main and resolve any conflicts systematically.

Your Submission

Submit through the OOP QM+ upload area. Ensure your code is IDE-agnostic and compiles using standard command-line tools without any special configuration.

- The textual version should be run by calling the startRace method located in the Part1 folder.
- The graphical version should be initiated using the startRaceGUI method located in the Part2 folder.

Your submission must include:

35. A Report in PDF format, named Report.pdf, providing detailed responses to Part I of the project.
36. A Zipped Working Directory named TypingRaceSimulator, including the .git folder and all source code for both parts.
37. A README File with clear step-by-step instructions to compile and run both the textual and graphical versions.

Assessment Criteria

Part I (60%)

Typist Class (40%)

38. Correctness of Typist Class Definition (20%)
    - **Fields and Constructor (10%):** All six required fields correctly defined. Constructor has the correct signature and initialises all fields appropriately.
    - **Methods (10%):** All required methods correctly implemented. Particular attention to the correct behaviour of `slideBack()`, `recoverFromBurnout()`, and `resetToStart()` – these involve non-trivial logic.
39. Encapsulation (5%)
    - Getters and setters correctly implemented with controlled access. Validation in `setAccuracy()` (clamping to 0–1) and `slideBack()` (clamping progress to 0) is present and correct.
40. Code Style and Conventions (5%)
    - Standard Java naming conventions, consistent indentation, and meaningful comments throughout.
41. Testing and Validation (10%)
    - **Comprehensive Testing (5%):** Tests cover all non-trivial cases including boundary conditions: progress at zero before a slideBack, burnout countdown, resetToStart clearing all state, accuracy clamping.
    - **Evidence of Testing (5%):** Screenshots or terminal output with clear explanations of what each test verifies and why it confirms correct behaviour.

TypingRace Class (20%)

42. Improvements to TypingRace Class (15%)
    - **Enhancements (10%):** Winner's name correctly displayed at the end of the race. Meaningful improvements to race logic, bug fixes implemented correctly.
    - **Code Quality (5%):** Improvements are well-structured, readable, and do not introduce new issues.
43. Bug Identification and Resolution (5%)
    - **Identifying Issues (3%):** Each bug is identified, clearly described, and its effect on the simulation is explained.
    - **Proposing Solutions (2%):** Clear, logically sound solutions or suggestions for each identified problem.

Part II (20%)

Criterion	Marks	Key Requirements
Interactive Race Configuration	5%	Passage selection, seat count, difficulty modifiers implemented and functional
Customisable Typists	5%	Typing style, keyboard type, symbol/colour, accessories with meaningful attribute impact
Statistics & Analytics	5%	WPM, accuracy %, burnout count, personal bests, historical data, comparison view

Reward System (Option A, B, or both)	5%	Leaderboard/ranking or sponsor/prize system fully implemented. Both options = higher marks.

Part III (20%)

Criterion	Marks	Key Requirements
Repository Setup & Structure	4%	Correct initialisation, TypingRaceSimulator root, Part1/Part2 folders, no clutter
GitHub & Remote Sync	4%	Private repo created, consistent and regular pushes throughout development
README Documentation	4%	Clear setup instructions, all dependencies listed, usage guidelines for both parts
Commit Quality & History	4%	Descriptive messages, frequent and logically structured commits
Branching & Merging	4%	gui-development branch used throughout Part II, clean merge with conflicts resolved

Marking Scale

For each criterion, work will be evaluated as follows:

Judgement	Mark	Description
Not attempted	0	No attempt was made.
Fail	1–3	Attempt made, but significant errors or omissions indicate insufficient understanding of the course material.
Basic	4–5	Overall correct work demonstrating a basic understanding of the material. May contain minor errors.
Good	6–7	Reflects deep understanding with independent thinking and application. Errors, if any, are minor and infrequent.
Excellent	8–9	Thorough understanding complemented by independent research and critical thinking. Minimal or no errors.
Exceptional	10	Surpasses first-year undergraduate expectations. Outstanding comprehension, analysis, and originality.